

Keywords: KV cache • sparse attention • long-context inference • attention compensation

CompKV

Compensation-Aware KV Selection for Long-Context LLM Inference

By Anonymous et al.

arXiv:2609.26300 • September 2026

By pairing block-level KV selection with mean compensation—which yields an exact log-partition identity—CompKV derives a selection priority proportional to block attention mass times within-block logit variation, consistently outperforming prior sparse-attention methods on RULER and LongBench-Pro at tight token budgets.

AT A GLANCE

Problem: Sparse-attention KV selection methods discard unselected context entirely, introducing approximation error that biases the softmax partition function.

Why it matters: Long-context tasks requiring global aggregation fail most under naive selection; more principled compensation preserves performance at aggressive memory budgets.

Method: Mean compensation for unselected blocks yields an exact log-partition identity; the resulting optimal selection priority is block attention mass $\times$ within-block logit variation.

Result: CompKV consistently beats selection-only and eviction-based sparse attention on Llama-3.1-8B and Qwen3 models.

Evidence: RULER and LongBench-Pro benchmarks at context lengths up to 128K tokens.

The Big Picture

Long-context LLM inference is memory-bound. Fitting a 128K-token context in a transformer requires gigabytes of key-value (KV) cache that exceed typical GPU memory. Sparse-attention methods reduce pressure by loading only a selected subset of KV blocks per query. The dominant selection heuristic—loading the top- K blocks by attention mass—has a hidden flaw: it treats unselected blocks as if they carry zero weight, silently biasing the softmax partition function and the attention output.

CompKV revisits this approximation and asks: given that some blocks will not be loaded, what is the best way to account for them? The answer is mean compensation—replacing each unselected block with its mean—combined

with a principled selection criterion derived from the compensation’s residual error.

Why Existing Approaches Fall Short

Eviction methods (H2O, StreamingLLM) permanently discard tokens, making them unrecoverable for later attention layers.

Selection without compensation loads the top- K blocks and zeroes the rest, which underestimates the softmax denominator whenever unselected blocks carry non-trivial mass.

KV quantization reduces per-token size but does not reduce the number of tokens read per attention call.

IO-aware kernels overlap memory transfers with compute but leave the approximation error unaddressed.

None of these methods reason about how unselected context should be summarised to correct the partition function.

Core Mechanism

Mean Compensation. For each unselected block j , CompKV replaces every logit with the block mean $\bar{e}_j$. This is the unique single-number summary that preserves the block’s contribution to the log-partition function to first order—a property that zero-padding and other approximations lack.

Compensation-Aware Priority. After fixing mean compensation, the residual approximation error for block j is bounded by the product of its attention mass m_j (how much it contributes to the output) and its within-block logit standard deviation σ_j (how much individual weights differ from the mean). CompKV selects the blocks with the largest $m_j \cdot \sigma_j$ —those for which mean compensation is most inadequate.

Technical Formulation

Let the KV cache be partitioned into blocks of size b . For block j , denote the attention logits $\mathbf{e}_j \in \mathbb{R}^b$ and the block mean $\bar{e}_j = \frac{1}{b} \sum_i e_{j,i}$.

The log-partition under mean compensation:

$$\log Z_j^{\text{approx}} = \bar{e}_j + \log b$$

which equals the exact value when all within-block logits are equal. The residual error is bounded:

$$\Delta_j \leq m_j \cdot \sigma_j$$

where $m_j = \text{softmax_mass}(\mathbf{e}_j)$ and $\sigma_j = \text{std}(\mathbf{e}_j)$. The CompKV selection priority score is:

$$s_j = m_j \cdot \sigma_j$$

The top- K blocks by s_j are loaded; the rest contribute via their mean.

Learning or Inference Procedure

CompKV is training-free and applies at inference time:

1. A fused GPU kernel computes per-block mass estimates and logit standard deviations using grouped variance projection of the query against pre-computed block mean keys.
2. Blocks are ranked by $s_j = m_j \cdot \sigma_j$; top- K are selected.
3. Selected blocks contribute via exact attention; unselected blocks contribute via their mean, with the partition function adjusted accordingly.

Implementation: BF16 KV cache and FP32 block summaries are kept in pinned CPU memory. A main CUDA stream handles selection-region transfer and exact-attention kernels; an auxiliary stream transfers mean values in parallel, eliminating blocking synchronization.

What the Guarantee Says

Mean compensation guarantees exact log-partition preservation (to first order) when within-block logits are uniform. For non-uniform blocks, the residual error is bounded by $m_j \cdot \sigma_j$ —precisely the quantity used for selection. Under a fixed budget of K blocks, CompKV minimises the total approximation error bound by choosing the blocks for which the bound is largest.

Experimental Findings

- Models: Llama-3.1-8B-Instruct, Qwen3-8B, Qwen3-32B
- Benchmarks: RULER (controlled retrieval and aggregation), LongBench-Pro (realistic long-context tasks), context lengths to 128K tokens
- CompKV outperforms selection-only (top- K by mass), H2O, and StreamingLLM variants consistently across all models and benchmarks
- Gains are largest on aggregation tasks and at tighter budgets (K is small), where the compensation correction matters most

Ablations and Interpretation

Selection criterion: Replacing $m_j \cdot \sigma_j$ with mass-only or variation-only degrades performance, confirming that the joint criterion is necessary.

Compensation type: Mean compensation outperforms zero compensation; the gap grows with block size (larger within-block variation).

IO overhead: The fused kernel architecture hides trans-

fer latency by overlapping streams; throughput is competitive with GPU-only sparse attention at high batch sizes.

Reference

Anonymous et al. **CompKV: Compensation-Aware KV Selection for Long-Context LLM Inference.** arXiv:2609.26300, September 2026.

<https://arxiv.org/abs/2609.26300>